

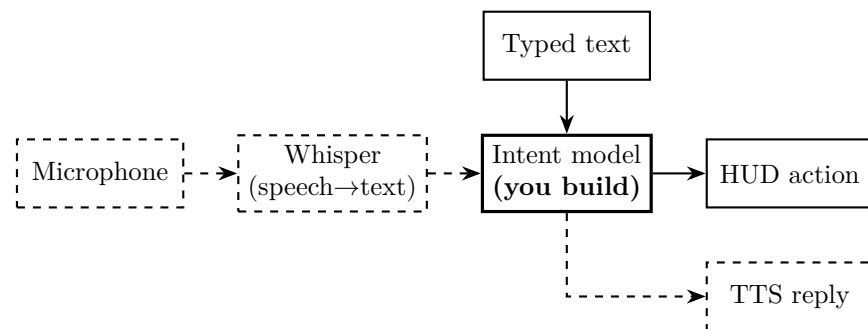
CLAWS Onboarding: EVA Voice Intent Model

CLAWS Voice Assistant Team

Fall 2026

Goal: write training sentences for 35 EVA intents, fine-tune a small model, classify sentences it has never seen. Text in, intent out is required. Voice (Whisper + TTS) is a stretch goal. Shared code is in `onboarding/code/`; your work goes in your own folder.

1 Where this fits



Solid = required. Dashed = stretch. The model picks an intent (plus a name for five intents). The HUD reads the actual value from telemetry.

2 The intents

Labels are **case-sensitive**. Copy them exactly. Also in `code/intents.json`.

Intent	Meaning	Name
<code>get_oxy_pri_storage</code>	Primary oxygen storage %	
<code>get_oxy_sec_storage</code>	Secondary oxygen storage %	
<code>get_oxy_pri_pressure</code>	Primary oxygen tank pressure	
<code>get_oxy_sec_pressure</code>	Secondary oxygen tank pressure	
<code>get_suit_pressure_oxy</code>	Suit O ₂ partial pressure	
<code>get_suit_pressure_co2</code>	Suit CO ₂ partial pressure	
<code>get_suit_pressure_other</code>	Suit trace gas pressure	
<code>get_suit_pressure_total</code>	Total suit pressure	
<code>get_helmet_pressure_co2</code>	Helmet CO ₂ pressure	
<code>get_fan_pri_rpm</code>	Primary fan speed (RPM)	
<code>get_fan_sec_rpm</code>	Secondary fan speed (RPM)	
<code>get_scrubber_a_co2_storage</code>	Scrubber A CO ₂ storage	
<code>get_scrubber_b_co2_storage</code>	Scrubber B CO ₂ storage	
<code>get_temperature</code>	Suit internal temperature	
<code>get_coolant_storage</code>	Coolant storage %	
<code>get_coolant_gas_pressure</code>	Coolant gas loop pressure	
<code>get_coolant_liquid_pressure</code>	Coolant liquid loop pressure	
<code>get_heart_rate</code>	Astronaut heart rate	
<code>get_oxy_consumption</code>	Oxygen consumption rate	
<code>get_co2_production</code>	CO ₂ production rate	
<code>get_eva_elapsed_time</code>	Elapsed mission time	
<code>get_primary_battery_level</code>	Primary battery %	
<code>get_secondary_battery_level</code>	Secondary battery %	
<code>get_warnings</code>	Active warnings and alerts	
<code>open_menu_vitals</code>	Open vitals panel	
<code>open_menu_navigation</code>	Open navigation / map panel	
<code>open_menu_tasks</code>	Open tasks / checklist panel	
<code>open_menu_uia</code>	Open UIA airlock panel	
<code>close_menu</code>	Close the active panel	
<code>Get_coordinates</code>	Coordinates of a named target	✓
<code>Set_navigation_target</code>	Navigate to a named place	✓
<code>reroute_navigation</code>	Compute a new route	
<code>Add_waypoint</code>	Add a named waypoint	✓
<code>Delete_waypoint</code>	Delete a named waypoint	✓
<code>Complete_task</code>	Mark a named task complete	✓

3 Setup

Any laptop works (CPU is fine; MPS/CUDA used if present). Python 3.9+. Your folder is already made:

Member	Folder
Aaron	onboarding/submissions/aaron/
Evan	onboarding/submissions/evan/
Raiana	onboarding/submissions/raiana/
Sam	onboarding/submissions/sam/
Utsav	onboarding/submissions/utsav/
Vivian	onboarding/submissions/vivian/

```
git checkout -b onboarding/<your-name>
cd onboarding
python3 -m venv .venv && source .venv/bin/activate
pip install torch transformers sentence-transformers scikit-learn
cd submissions/<your-name>          # run everything from here
```

Each folder has an empty `data/train.jsonl`, an empty `runs/`, and a `notes.md` template. Run scripts from your folder as `python ../../code/<script>.py`; data and results land in your folder. Don't copy the code into your folder.

4 Write the training data

The model is only as good as your sentences. One file, `data/train.jsonl`, one JSON object per line, exactly one intent each (same format as `EVA/data/`):

```
{"text": "what's my primary oxygen at", "intents": ["get_oxy_pri_storage"]}
{"text": "O2 secondary tank pressure?", "intents": ["get_oxy_sec_pressure"]}
{"text": "uh how hot is it in the suit", "intents": ["get_temperature"]}
{"text": "pull up the map", "intents": ["open_menu_navigation"]}
{"text": "drop a waypoint called Crater Rim", "intents": ["Add_waypoint"]}
```

Rules

- **30+ per intent** (50 ideal). Write them yourself; LLM ideas OK if you check every line.
- Mix questions, commands and clipped phrases (“fan RPM?”). 2 to 20 words.
- Vary words: oxygen/O2/ox, battery/charge/power, menu/panel/screen.
- Sound like speech: “uh”, “please”, lowercase, no punctuation.
- Always say which one: primary/secondary, A/B, gas/liquid.
- Vary names in the five name intents. No near-duplicates.

Listing 1: `validate.py`

```
"""Check a training file: known labels, one label per row, no duplicates, enough rows."""
import json
import sys
from collections import Counter
from pathlib import Path

MIN_PER_INTENT = 30

labels = {x["intent"] for x in json.load(open(Path(__file__).with_name("intents.json")))}
seen, counts, problems = set(), Counter(), 0

for n, line in enumerate(open(sys.argv[1], encoding="utf-8"), 1):
    if not line.strip():
        continue
    row = json.loads(line)
    text, intents = row["text"].strip(), row["intents"]
    if len(intents) != 1 or intents[0] not in labels:
        print(f"line {n}: bad intents {intents}")
        problems += 1
    if text.lower() in seen:
        print(f"line {n}: duplicate text '{text}'")
        problems += 1
    seen.add(text.lower())
    counts.update(intents)

for label in sorted(labels):
    if counts[label] < MIN_PER_INTENT:
        print(f"{label}: only {counts[label]} rows (need {MIN_PER_INTENT})")
```

```
        problems += 1

print(f"{sum(counts.values())} rows, {problems} problems")
sys.exit(1 if problems else 0)
```

```
python ../../code/validate.py data/train.jsonl    # fix until "0 problems"
```

5 Build the model

Step 0: baseline. Frozen sentence embeddings + logistic regression. This is the number to beat.

Listing 2: baseline.py

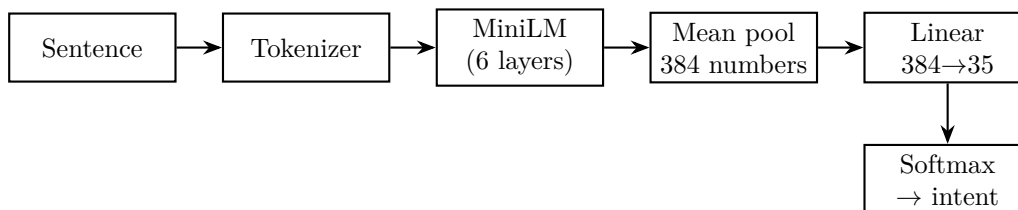
```
"""Step 0: frozen sentence embeddings + logistic regression. No fine-tuning."""
import json
from sentence_transformers import SentenceTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split

rows = [json.loads(line) for line in open("data/train.jsonl") if line.strip()]
texts = [r["text"] for r in rows]
labels = [r["intents"][0] for r in rows]
X_tr, X_te, y_tr, y_te = train_test_split(
    texts, labels, test_size=0.15, stratify=labels, random_state=0)

encoder = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
clf = LogisticRegression(max_iter=2000).fit(encoder.encode(X_tr), y_tr)
print(classification_report(y_te, clf.predict(encoder.encode(X_te)), zero_division=0))
```

```
python ../../code/baseline.py > baseline.txt
```

Step 1: fine-tune. MiniLM (22M params) + mean pooling + one linear layer, same as the club model in EVA/Models/singleintentmodel/.



Listing 3: model.py

```
"""MiniLM encoder + mean pooling + one linear layer (same shape as EVA/Models/
singleintentmodel)."""
import torch
from torch import nn
from transformers import AutoModel

MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"

class IntentModel(nn.Module):
    def __init__(self, num_labels):
        super().__init__()
        self.encoder = AutoModel.from_pretrained(MODEL_NAME)
        self.head = nn.Linear(self.encoder.config.hidden_size, num_labels)
```

```

def forward(self, batch):
    hidden = self.encoder(**batch).last_hidden_state # (B, tokens, 384)
    mask = batch["attention_mask"].unsqueeze(-1).float()
    pooled = (hidden * mask).sum(1) / mask.sum(1).clamp(min=1e-9)
    return self.head(pooled) # (B, num_labels) logits

def best_device():
    if torch.backends.mps.is_available():
        return torch.device("mps")
    if torch.cuda.is_available():
        return torch.device("cuda")
    return torch.device("cpu")

```

`train.py` holds out 15%, trains 10 epochs, prints accuracy each epoch, then lists every miss. Expect >0.95 and a win over the baseline; if not, fix your data.

Listing 4: `train.py`

```

"""Step 1: fine-tune MiniLM + linear head on data/train.jsonl, save to runs/."""
import json
import os
import random

import torch
from sklearn.metrics import accuracy_score, f1_score
from sklearn.model_selection import train_test_split
from transformers import AutoTokenizer

from model import MODEL_NAME, IntentModel, best_device

EPOCHS, BATCH, LR = 10, 16, 5e-5
random.seed(0)
torch.manual_seed(0)
device = best_device()

rows = [json.loads(line) for line in open("data/train.jsonl") if line.strip()]
labels = sorted({r["intents"][0] for r in rows})
label2id = {label: i for i, label in enumerate(labels)}
texts = [r["text"] for r in rows]
ys = [label2id[r["intents"][0]] for r in rows]
X_tr, X_te, y_tr, y_te = train_test_split(
    texts, ys, test_size=0.15, stratify=ys, random_state=0)

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = IntentModel(len(labels)).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=LR)

def encode(batch_texts):
    enc = tokenizer(batch_texts, padding=True, truncation=True,
                    max_length=64, return_tensors="pt")
    return {k: v.to(device) for k, v in enc.items()}

```

```

def predict_ids(batch_texts):
    model.eval()
    with torch.no_grad():
        return [i for s in range(0, len(batch_texts), BATCH)
                for i in model(encode(batch_texts[s:s + BATCH])).argmax(1).tolist()]

for epoch in range(1, EPOCHS + 1):
    model.train()
    order = list(range(len(X_tr)))
    random.shuffle(order)
    for s in range(0, len(order), BATCH):
        idx = order[s:s + BATCH]
        logits = model(encode([X_tr[i] for i in idx]))
        target = torch.tensor([y_tr[i] for i in idx], device=device)
        loss = torch.nn.functional.cross_entropy(logits, target)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    pred = predict_ids(X_te)
    print(f"epoch {epoch}  loss {loss.item():.3f}  "
          f"acc {accuracy_score(y_te, pred):.3f}  "
          f"macro-F1 {f1_score(y_te, pred, average='macro'):.3f}")

# Show every mistake on the held-out set: this is where your data needs work.
for text, gold, guess in zip(X_te, y_te, pred):
    if gold != guess:
        print(f"MISS  '{text}'  expected {labels[gold]}  got {labels[guess]}")

os.makedirs("runs", exist_ok=True)
torch.save(model.state_dict(), "runs/model.pt")
json.dump(label2id, open("runs/label2id.json", "w"), indent=2)
json.dump({"accuracy": accuracy_score(y_te, pred),
          "macro_f1": f1_score(y_te, pred, average="macro"),
          "train_rows": len(X_tr), "test_rows": len(X_te),
          "epochs": EPOCHS, "batch": BATCH, "lr": LR},
          open("runs/metrics.json", "w"), indent=2)

```

```
python ../../code/train.py
```

6 Evaluate and predict

Listing 5: predict.py

```
"""Load runs/model.pt and classify text. Usage: python predict.py "open the nav menu" """
import json
import re
import sys

import torch
from transformers import AutoTokenizer

from model import MODEL_NAME, IntentModel

label2id = json.load(open("runs/label2id.json"))
labels = sorted(label2id, key=label2id.get)
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = IntentModel(len(labels))
model.load_state_dict(torch.load("runs/model.pt", map_location="cpu"))
model.eval()

SLOT_INTENTS = {"Get_coordinates", "Set_navigation_target",
                "Add_waypoint", "Delete_waypoint", "Complete_task"}
# Naive: the slot is whatever follows the last trigger word. Improve this.
TRIGGER = re.compile(r"\b(?:to|named|called|for|of|waypoint|task)\s+", re.I)

def extract_slot(text):
    parts = TRIGGER.split(text.strip().rstrip("?!"))
    return parts[-1].strip() if len(parts) > 1 else None

def predict(text):
    with torch.no_grad():
        probs = model(tokenizer([text], return_tensors="pt")).softmax(1)[0]
        conf, idx = probs.max(0)
        result = {"text": text, "selected_intent": labels[idx.item()],
                  "confidence": round(conf.item(), 3)}
        if result["selected_intent"] in SLOT_INTENTS:
            result["slot"] = extract_slot(text)
    return result

if __name__ == "__main__":
    print(json.dumps(predict(" ".join(sys.argv[1:])))
```

```
python ../../code/predict.py "add a waypoint named Olympus Ridge"
{"text": "add a waypoint named Olympus Ridge", "selected_intent": "Add_waypoint",
 "confidence": 0.969, "slot": "Olympus Ridge"}
```

Held-out accuracy is not the real test. Write 10+ new sentences (ideally a friend writes some) and see what breaks. Common confusions: primary/secondary, scrubber A/B, coolant gas/liquid, open_menu_navigation/reroute_navigation, and the coordinate/target/waypoint intents. Fix

confusions with better data, not code.

Names: `extract_slot` takes the text after the last trigger word. It's naive ("mark task soil sample complete" → `soil sample complete`). Improving it is a bonus.

7 Stretch: make it talk

```
pip install faster-whisper sounddevice pyttsx3
python ../../code/voice.py
```

Listing 6: voice.py

```
"""Stretch goal: microphone -> Whisper -> intent model -> spoken reply.
pip install faster-whisper sounddevice pyttsx3"""
import sounddevice as sd
import pyttsx3
from faster_whisper import WhisperModel

from predict import predict

RATE, SECONDS, MIN_CONF = 16000, 4, 0.6
# Domain words help Whisper spell things the way your training data does.
HINT = "EVA, UIA, CO2, O2, scrubber, coolant, waypoint, RPM, HUD"

stt = WhisperModel("base.en", device="cpu", compute_type="int8")
tts = pyttsx3.init()

while True:
    input("Press Enter, then speak for 4 seconds...")
    audio = sd.rec(RATE * SECONDS, samplerate=RATE, channels=1, dtype="float32")
    sd.wait()
    segments, _ = stt.transcribe(audio.flatten(), language="en", initial_prompt=HINT)
    text = " ".join(s.text for s in segments).strip()
    if not text:
        continue
    result = predict(text)
    print(result)
    if result["confidence"] < MIN_CONF:
        reply = "Sorry, say that again."
    else:
        reply = result["selected_intent"].replace("_", " ")
    tts.say(reply)
    tts.runAndWait()
```

Next steps: `small.en` Whisper, speak the real value back, push-to-talk.

8 What to hand in

One big PR. Commit as often as you like on your branch, but open a single PR when everything below is done. No partial PRs. Only touch your own folder, so the six PRs never conflict.

- ☐ `data/train.jsonl` (0 problems in `validate.py`)
- ☐ `runs/metrics.json`, `runs/label2id.json`
- ☐ `baseline.txt` (output of `baseline.py`)
- ☐ `notes.md`: filled in (results, 10+ unseen sentences, what failed)
- ☐ **Not** `runs/model.pt` (`.gitignore` skips it)

Stuck for over an hour? Ask in the channel.